

Qiskit Summer Dojo: Qiskit v2.X資格対策 #4

~量子回路の結果の取得と分析~

~OpenQASMによる操作~

2026/09/03

Hotaka Hayashi

おことわり

「Qiskit v2.x Developer Associate – Practice Questions」に掲載されている内容について、重要ポイントの紹介とそれに紐づく問題の解説を行います。

- 問題はこちら: <https://quantum-tokyo.github.io/qiskit-certification-prep/>
- 例題を出した時に少し時間を取りますので、正解がどれかを考えてみてください。
- 本セッションでは時間の関係で全ての問題の解説は行いませんので、ご了承ください。

Qiskit v2.X developer
certification exam prep

Sample Test

Section1: Perform Quantum
Computing

Section 2: Visualize quantum
circuits, measurements, and
states

Section 3: Create quantum
circuits

Section 4: Run quantum circuits

Section 5-6: Use the
sampler/estimator primitive

Section 7: Retrieve and analyze
the results of quantum circuits

Section 8: Operate with
OpenQASM

Qiskit v2.x Developer Associate – Practice Questions

Daiki Murata

Quantum Tokyo

Web page is [here](#) !



CONTENTS ▾

- 🎯 Purpose (What & For Whom)
- 📄 Exam Overview
- 📁 Exam Domains
- ⚠️ Disclaimer
- 🔗 Official Resources



目次

Section7: 量子回路の結果の取得と分析

- TASK7.1 : 過去の実行結果の取得
- TASK7.2 : ジョブの監視

Section8: OpenQASMによる操作

目次

Section7: 量子回路の結果の取得と分析

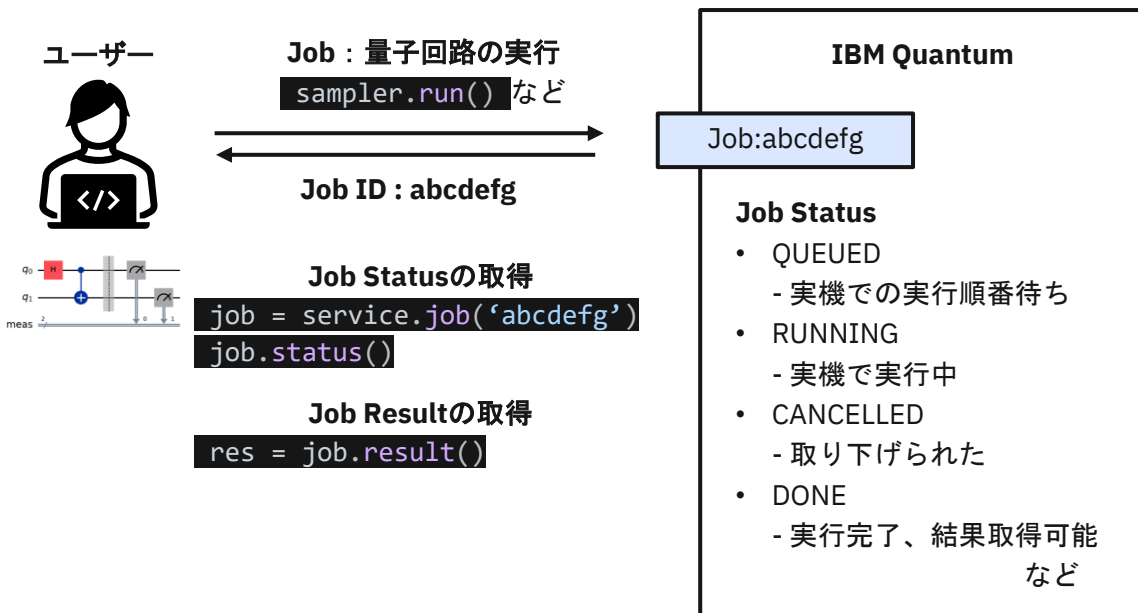
- TASK7.1 : 過去の実行結果の取得
- TASK7.2 : ジョブの監視

Section8: OpenQASMによる操作

Section7 : 量子回路の結果の取得と分析 Overview

IBM Quantumの実機で量子回路を実行したとき、そのステータスや結果は**Job**というオブジェクトに紐づけられている。

Jobには一意の**Job ID**が割り振られ、これをもとにステータスや結果などの情報を取得する。



Status and usage

Status

✓ Completed

Total completion time

2,819m 7s

Total usage

5s

✓ Created: Aug 16, 2026, 9:49 PM

✓ Pending: 2,817m 2s

✓ In progress: Aug 18, 2026, 8:46 PM
Usage: 5s

✓ Completed: Aug 18, 2026, 8:48 PM

IBM Quantum Platform上での表記

Section7 : 量子回路の結果の取得と分析 Overview

IBM Quantum Platform上で、過去のJobやその状況を一覧することができる。

My recent workloads					View all
ID	Status	Instance	Created ↓	QPU	Usage
da[REDACTED]s7...	✓ Completed	[REDACTED]	8/16/26, 9:49 PM	ibm_fez	5s
da[REDACTED]7...	✗ Canceled	[REDACTED]	8/16/26, 2:35 PM	ibm_fez	0s
d9[REDACTED]73...	✓ Completed	[REDACTED]	7/26/26, 3:31 PM	ibm_marrakesh	4m 25s
d9[REDACTED]8e...	✓ Completed	[REDACTED]	7/26/26, 3:30 PM	ibm_marrakesh	16s
d9[REDACTED]89...	✓ Completed	[REDACTED]	7/26/26, 3:29 PM	ibm_marrakesh	12s
d9[REDACTED]8c...	✓ Completed	[REDACTED]	7/26/26, 3:28 PM	ibm_marrakesh	12s
d9[REDACTED]3...	✓ Completed	[REDACTED]	7/26/26, 2:58 PM	ibm_marrakesh	7s

IBM Quantum Platform上のJob一覧

TASK7.1 : 過去の実行結果の取得と分析 (Tips)

Jobの結果取得はJob ID指定のみでなく、オプションを付与して複数件取得したり、jsonとして出力できる。

通常の仕事結果の取得

```
job = service.job('abcdefg')  
res = job.result()
```

Jobは PrimitiveResultというオブジェクトとして得る。
Sampler, Estimatorの結果のみでなく、メタデータ等も含まれている。

Job結果をjsonへ保存・取得

```
json.dump(result, cls=RuntimeEncoder)  
json.load(file, cls=RuntimeEncoder)
```

qiskit_ibm_runtime.RuntimeEncoderを指定し、
jsonファイルへの出力・後でロードができる。

Jobの検索方法

- ・ タグ指定での取得

```
service.jobs(program_id="sampler", tags=["abc"])
```

- ・ 期間指定の取得

```
service.jobs(created_after=datetime.now()-timedelta(days=90))
```

TASK7.1 クイズ

10秒

6. Which statement about **job IDs** is most accurate for retrieving results later?
- a. You must keep or re-discover the job ID; retrieval later requires `service.job(<JOB_ID>)` before calling `.result()`.
 - b. Job IDs are optional; the service can always infer the correct job from the circuit metadata alone.
 - c. Job IDs expire immediately upon completion, so you cannot use them after the job finishes.
 - d. Job IDs can be reconstructed deterministically from the circuit hash and the backend name.

answer



TASK7.1 クイズ

6. Which statement about **job IDs** is most accurate for retrieving results later?

- a. You must keep or re-discover the job ID; retrieval later requires `service.job(<JOB_ID>)` before calling `.result()`.
- b. Job IDs are optional; the service can always infer the correct job from the circuit metadata alone.
- c. Job IDs expire immediately upon completion, so you cannot use them after the job finishes.
- d. Job IDs can be reconstructed deterministically from the circuit hash and the backend name.

answer

The answer is a.

The documentation emphasizes that retrieving results later relies on the job ID. Save it at submission time (`job.job_id()`) or look it up on the Workloads page or with `service.jobs(...)`.

ポイント :

- ・ Job IDは、Jobに対し一意に割り振られる。
- ・ Job IDを使用して、実行されたJobの結果を後から検索する。

TASK7.1 クイズ

10秒

2. You want to programmatically find jobs run in roughly the last three months to resume analysis. Which approach is correct?

- a. `service.jobs(created_after=datetime.now()-timedelta(days=90))` to list recent jobs and then select the ones you need.
- b. `service.jobs(last_three_months=True)` to let the service infer the period automatically.
- c. `service.list_recent_jobs(90_days=True)` and then call `.result()` on the list.
- d. `service.jobs()` always returns only the last 7 days, so older jobs cannot be listed.

TASK7.1 クイズ

2. You want to programmatically find jobs run in roughly the last three months to resume analysis. Which approach is correct?

- a. `service.jobs(created_after=datetime.now()-timedelta(days=90))` to list recent jobs and then select the ones you need.
- b. `service.jobs(last_three_months=True)` to let the service infer the period automatically.
- c. `service.list_recent_jobs(90_days=True)` and then call `.result()` on the list.
- d. `service.jobs()` always returns only the last 7 days, so older jobs cannot be listed.

answer

The answer is a.

`QiskitRuntimeService.jobs` accepts filters such as `created_after` (a `datetime.datetime` object). You can compute a boundary like "now - 90 days", list those jobs, and then retrieve any job by ID to get its results. :::

ポイント：

- ・ 期間指定でjob一覧を取得可能
- ・ オプションは自由に選べる方が使いやすい

TASK7.2 : ジョブの監視 Tips

Qiskitを用いたプログラムで、現在のJobのStatusを取得するコードや、Statusが終了状態になるまで待機するコードが存在する。

Job Status取得

```
job.status()
```

Job終了まで待機

```
job.wait_for_final_state()
```

Job Status

よく見る

- QUEUED
 - 実機での実行順番待ち
- RUNNING
 - 実機で実行中
- CANCELLED
 - 取り下げられた
- DONE
 - 実行完了、結果取得可能

あまり見ない

- ERROR
- INITIALIZING
- VALIDATING

TASK7.2 クイズ

10秒

7. Which job state typically appears **before** a job transitions to **RUNNING**?

- a. **QUEUED**
- b. **DONE**
- c. **ERROR**
- d. **CANCELLED**

answer



TASK7.2 クイズ

10秒

7. Which job state typically appears **before** a job transitions to **RUNNING**?

- a. **QUEUED**
- b. **DONE**
- c. **ERROR**
- d. **CANCELLED**

answer

The answer is a.

The job lifecycle often transitions from **QUEUED** → **RUNNING** → **DONE**. Errors or cancellations can occur at any stage. ...

ポイント :

- ・ Jobはまず実機実行の順番待ちキューに入り、順番が来たら実機で実行される

TASK7.2 クイズ

10秒

2. When calling `job.wait_for_final_state()`, what happens if the job is still queued or running?
- a. It raises an error immediately.
 - b. It blocks until the job reaches a terminal state (such as DONE, ERROR, or CANCELLED).
 - c. It prints logs continuously until the job finishes.
 - d. It returns partial results at intervals.

answer



TASK7.2 クイズ

10秒

2. When calling `job.wait_for_final_state()`, what happens if the job is still queued or running?
- a. It raises an error immediately.
 - b. It blocks until the job reaches a terminal state (such as DONE, ERROR, or CANCELLED).
 - c. It prints logs continuously until the job finishes.
 - d. It returns partial results at intervals.

answer

The answer is b.

`wait_for_final_state()` suspends execution until the job is completed, failed, or cancelled. This is useful in scripts where interactive monitoring is not required.

ポイント :

- ・ `final_state` とは何か考えてみる
→ 必ずしも正常終了ではない！

目次

Section7: 量子回路の結果の取得と分析

- TASK7.1 : 過去の実行結果の取得
- TASK7.2 : ジョブの監視

Section8: OpenQASMによる操作

Section8: OpenQASMによる操作 Overview

OpenQASMはIBM QPUと互換性がある、量子回路を記述するための言語である。

OpenQASM (open quantum assembly language), a machine-independent programming interface compatible with IBM® QPUs, is an imperative programming language for describing quantum circuits. OpenQASM uses the quantum circuit model to express quantum programs as ordered sequences of parameterized operations (such as gates, measurements, and resets) and real-time classical computation. In addition to quantum algorithms, OpenQASM can describe circuits intended to characterize, validate, or debug quantum processors.

As the needs of QPU development have evolved, OpenQASM's feature list has expanded in response; the latest version, [OpenQASM 3](https://quantum.cloud.ibm.com/docs/en/guides/introduction-to-qasm), incorporates extensions including classical feed-forward flow control, gate modifiers, and pulse implementations.

<https://quantum.cloud.ibm.com/docs/en/guides/introduction-to-qasm>

AIによるサマリ：

- IBM QPUと互換性のあるアセンブリ言語で、量子回路を順序付けられたパラメータ化操作（ゲート、測定、リセット）と古典計算として記述します。
- Qiskit SDKは**OpenQASM**と**QuantumCircuit**クラス間の相互変換機能を提供し、ユーザーが複数の表現形式を柔軟に利用できます。

→試験では「こんな機能があれば便利だな」に当てはまるような選択肢を探せば良いイメージ
OpenQASM 2→OpenQASM 3のバージョンアップで機能が大幅増加したこともポイント

TASK8 クイズ

10秒

1. What is the main purpose of OpenQASM 3's *explicit timing* feature?

- a. To describe intended instruction scheduling independent of exact calibrated durations, enabling patterns like dynamical decoupling at the gate level
- b. To force every gate to specify a fixed duration known at compile time
- c. To disable all delay instructions so that gates execute as fast as possible
- d. To ensure gates are always scheduled strictly sequentially without overlap

answer



TASK8 クイズ

1. What is the main purpose of OpenQASM 3's *explicit timing* feature?

- a. To describe intended instruction scheduling independent of exact calibrated durations, enabling patterns like dynamical decoupling at the gate level
- b. To force every gate to specify a fixed duration known at compile time
- c. To disable all delay instructions so that gates execute as fast as possible
- d. To ensure gates are always scheduled strictly sequentially without overlap

answer

The answer is a.

Explicit timing lets authors express the design intent of scheduling without tying it to particular calibration durations, which enables techniques such as dynamical decoupling while still describing the circuit at the gate level. The other options misstate or restrict the timing model.

ポイント :

- ・ explicit timing(時間の明示)から想像される、あったら嬉しい機能は？

→ゲート実行時間のスケジューリングは、Dynamical Decoupling等で便利に使える。

END